

PRECEDENCE — lien priority settled by proven source-block position

Track: RWA · Built on the Attestcoin Protocol (`0x0FD2` + `0x0FD3`) on Creditcoin CC3, proving transactions on Ethereum Sepolia · deployed and settling on testnet

Competing lenders lock capital against real-world collateral on Ethereum Sepolia. The Attestcoin Native Query Verifier precompile at `0x0FD2` on Creditcoin CC3 proves each lock **and its exact position inside its source block**, so lien seniority settles by cryptographic proof instead of by whose paperwork reached a registry first. The winning claim mints as a tradeable ERC-1155 position, outpaced capital auto-refunds in full, and the encumbrance registry populates itself as a by-product of financing.

The one sentence that explains why this needs Attestcoin and not an oracle: an oracle can *assert* which lock came first, but it cannot *cryptographically commit* to that ordering — and without committed ordering, priority collapses.

Verify every claim below in five minutes

What	Where
Live app	https://precedence-beige.vercel.app
The app reports its own provenance	https://precedence-beige.vercel.app/api/config → <code>"mode": "chain"</code> , both chains <code>live: true</code>
Source (public)	https://github.com/ChaseBP/precedence
Demo video	https://www.youtube.com/watch?v=aFu5O5Mh-SQ
The integration, in one file	<code>contracts/src/creditcoin/AttestationGate.sol</code> — 342 lines
Integration write-up	<code>ATTESTCOIN_INTEGRATION.md</code> — every use of the protocol, with the code for each

```
# 10 live checks against BOTH Attestcoin precompiles → evidence/precompile.json
bun run ops/verify-precompile.ts

# the entire proof path against three real, pre-existing Sepolia transactions.
# Runs with NONE of our contracts deployed, so it isolates the protocol from our code.
cd worker && bun install && bun run src/cli.ts probe \
  0x144bc395f407d98d722e780d805786125e07283e141006c6c9bfafa64c5f6f78 \
  0xc58e9177d4dea9a25dc205d80e458da11efb56635f73900e2ae413ad699f77cf \
  0xf2b901f9898354cfbb01af86150105eceb1cc04d93a5365066f12fd3a6b6bd09

# the security controls — one passing rejection per attack
cd contracts && make test          # 98 tests
cd worker      && bun test          # 20 tests
cd precedence && bun test          # 995 tests

# the pre-submission gate: every documented URL resolves, every 0x literal is real or SAMPLE
bun run ops/check-submission.ts
```

The problem, and why 2025 made it urgent

Secured lending runs on **priority**: when collateral fails, who is paid first. The *invisible encumbrance problem* destroys that structure, because lenders cannot see each other's claims against the same asset.

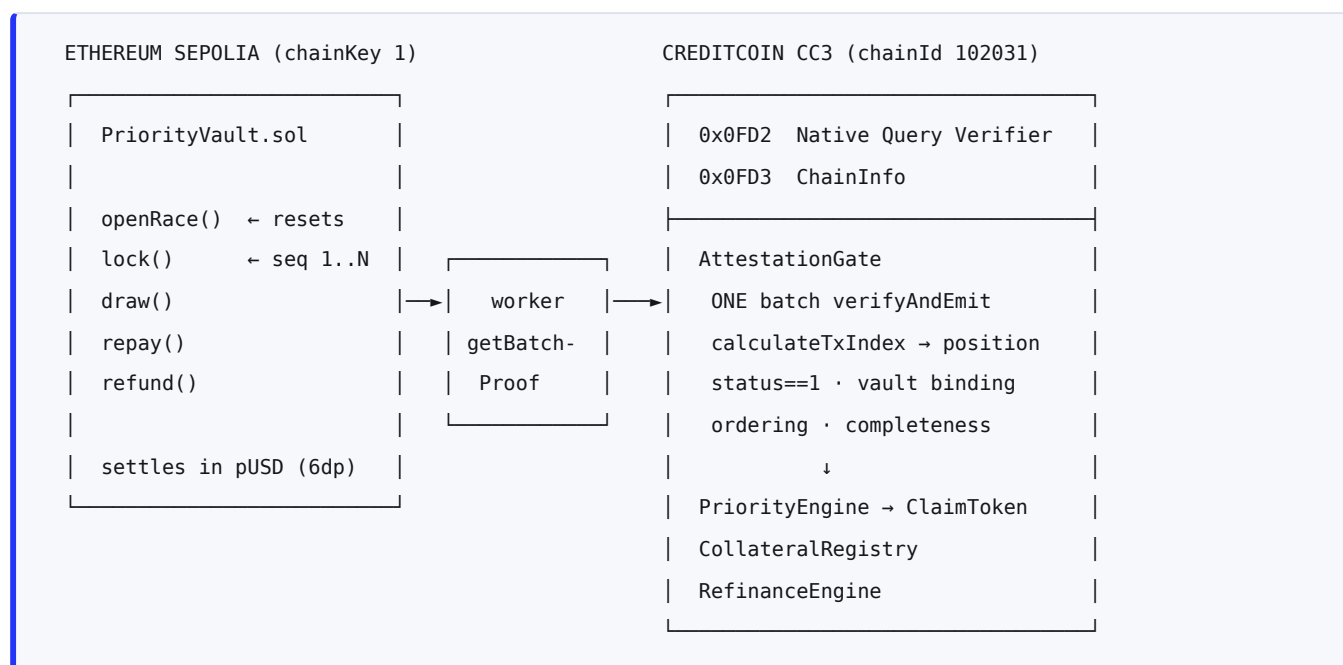
This is a live 2025 problem, not a historical curiosity:

- **Tricolor Holdings** filed Chapter 7 in September 2025 having pledged the same auto loans to roughly twenty lenders — about **\$1.4bn of real collateral against \$2.2bn pledged**, with roughly **\$500m of losses** landing at JPMorgan, Fifth Third and Barclays.
- **First Brands Group** filed Chapter 11 two weeks later, after selling the same receivables to multiple lenders.

The industry's answer has been **detection** — hash-based duplicate-check registries a lender *may* query before lending. That approach is real and it works (MonetaGo won distribution over SWIFT), but it is advisory, opt-in and bank-shaped, and it stopped neither 2025 collapse.

PRECEDENCE is not another detection service. The financing mechanism itself populates the registry. Using the system *is* joining it, and priority settles without anyone querying anything. There is no cold-start problem, because the first financing creates the first record.

How it works



1. An obligor registers collateral on Creditcoin as a **hash-unique** record. The document hash is `keccak256` over canonical JSON of `{assetType, docIdentifier, custodian, obligor, faceValueUsd}`, so one document cannot be registered twice under two identifiers.
2. Lenders **lock pUSD** on Sepolia against it, each declaring a tranche. Every lock is a real transaction at a real index in a real block, and the vault stamps a monotonic `seq`.
3. The worker waits for Attestcoin to attest the source block — **6.5-9.3 minutes, measured** — then builds **one shared continuity proof** for the whole race and preflights it for free.
4. `AttestationGate` settles the race in **one Creditcoin transaction**: a single batch `verifyAndEmit`, then the checks the precompile does not perform.
5. **Priority settles by proven** (`blockHeight, txIndex`) **ordering**. A declared tranche is a *preference*; proven ordering decides who wins it. Capital beyond a tranche's sizing is **auto-refunded in full, never silently demoted** — a lender who bid SENIOR did not consent to subordinate risk. Demotion exists, but only with that lender's explicit consent.
6. Repayment runs a **strict seniority waterfall**: rank 1 is satisfied completely before rank 2 receives a cent, so first loss lands on the most subordinate holder. Default runs a **deterministic unwind** — freeze, coverage stabilisation, grace, Dutch auction, default — where every transition is a permissionless, timestamp-gated keeper poke, so lender protection does not depend on our uptime.

Depth of Attestcoin utilisation

The protocol is not referenced; it is **load-bearing in seven distinct ways**. Remove any one and either the mechanism disappears or a real attack opens.

#	Use of the protocol	Why it is load-bearing	Code
1	<code>calculateTxIndex(merkleProof)</code> at <code>0x0FD2</code>	Sepolia's ~12s blocks make same-block locks likely, and block height alone cannot order them. The index is derived from the Merkle path , so the tie-break is <i>proven</i> , not asserted. Without this the protocol has no mechanism at all.	<code>AttestationGate.sol:195, 249</code>
2	On-chain batch <code>verifyAndEmit</code> — <code>uint64[] heights +</code> <code>bytes[] + MerkleProof[]</code> + one shared <code>ContinuityProof</code>	An entire race settles in a single on-chain call — atomic <i>by construction</i> , not atomic because we wrapped N calls in a transaction. There is no window in which half a race is settled.	<code>AttestationGate.sol:143</code>
3	Ordering verified on-chain , never trusted from the prover	The gate re-derives every index itself and requires strictly increasing <code>(height, txIndex)</code> . A prover cannot present a race in a self-serving order.	<code>PriorityProofLib.sol</code>
4	Vault binding — the control the reference pattern omits	<code>ASCBASE</code> computes its replay key as <code>keccak256(chainKey, height, txIndex)</code> and never records which contract emitted the verified transaction. So an attacker deploys a look-alike contract on Sepolia, emits an identical <code>Lock</code> event for free, obtains a genuine Attestcoin proof, and takes senior priority against collateral they never funded. We bind every proof to the collateral's registered vault via <code>EvmV1Decoder.LogEntry.address_</code> .	<code>PriorityProofLib.sol</code>
5	<code>status == 0x1</code> enforced by the dApp	The precompile proves inclusion , not success. A reverted lock is a genuinely included transaction with a real block and index — without this check a lender could deliberately revert their lock, escrow nothing, and still hold a valid inclusion proof for senior priority.	<code>PriorityProofLib.sol</code>
6	Completeness via the vault's own <code>seq</code>	Ordering alone is not enough: a prover who submits a <i>subset</i> can promote a friend by omitting the lock that beat them. <code>openRace()</code> resets the counter, and the gate requires <code>seq</code> contiguous from 1 — which makes omitting a <i>middle</i> lock impossible, and the start-at-1 requirement catches omission of the <i>first</i> lock, the one that actually pays. Tail truncation passes and is self-defeating: it only drops later, more-junior locks whose holders can prove their own.	<code>AttestationGate.sol</code>
7	<code>0x0FD3</code> ChainInfo, for honest waiting	<code>get_supported_chains</code> confirms Sepolia is chainKey 1 <i>from the chain itself</i> ; <code>get_latest_attestation_height_and_hash</code> and <code>is_height_attested</code> are how the interface reports the remaining wait as a live block countdown instead of hiding it behind a spinner.	<code>precedence/sdk/src/attestcoin.ts</code>

A detail worth naming, because it cost us time: `0x0FD3` is a Substrate **runtime** precompile, so its methods are `snake_case` rather than the Solidity convention. Calling `getLatestAttestationHeightAndHash` returns empty data that decodes as **zero** — which reads as "never attested" and is the most misleading failure available.

Also verified rather than assumed: the batch overload of `verifyAndEmit` is easy to conclude is absent, because the published interface listing does not make it obvious. We pinned `@gluwa/usc-contracts@0.2.0`, read the interface source, and probed `0x0FD2` directly — the batch selector reverts with `"Continuity chain cannot be empty"`, a validation error raised *from inside the implementation*,

which is only reachable if the selector decoded and ran. That is proof the batch entry point exists on the deployed precompile, and it is the one we use.

SDK surface: `@gluwa/usc-sdk@0.18.0`, `@gluwa/usc-contracts@0.2.0`, Solidity `^0.8.28`. `via_ir = true` is **required, not preferred** — the batch signature exhausts the EVM stack in the legacy pipeline and the contracts will not compile without it. Batch limits respected and checked before a proof is built: `MAX_BATCH_SIZE` 10 transactions, `MAX_BATCH_RANGE` 1000 blocks.

Proven on chain: the same-block settlement

This is the case the whole design exists for, and it happened on testnet. Record: `evidence/race-88857090.json`.

Four lenders locked against collateral

`0x88857090bc93b1a1de83c7f4838af8684ddd9d61d4e865d431f2e6df3641db3b`. **Two of the locks landed in the same Sepolia block, 11626711** — so the block number cannot order them.

Lender	Sepolia block	txIndex	Outcome
<code>0x537fc713...</code>	11626711	71	SENIOR , \$5,100
<code>0x8E83a933...</code>	11626711	72	JUNIOR , \$2,550
<code>0x4f3F6012...</code>	11626712	58	refunded in full
<code>0x78a5C38c...</code>	11626713	64	refunded in full

Same block. Index 71 takes senior; index 72 takes junior. **Nobody decided that — the precompile derived it.**

Attestation wait	6.37 minutes
Settled on Creditcoin	tx <code>0x1a61728c7fa29f1ec97736c7e4cbdbf4f0659ddf79e5c924d96ef60152163870</code> , block 5423422
Gas used	1,177,620 — one transaction for a four-lock race, ~1.6% of a CC3 block
Independent cross-check	<code>calculateTxIndex</code> vs the block explorer, for all four locks: agrees

The cross-check is the point. The precompile derives each position from a Merkle path; Etherscan reports it from its own index of the chain. **Every settlement records whether the two agree**, and a disagreement would be a finding rather than something to paper over.

Explorer links:

- Creditcoin settlement: <https://creditcoin-testnet.blockscout.com/tx/0x1a61728c7fa29f1ec97736c7e4cbdbf4f0659ddf79e5c924d96ef60152163870>
- Sepolia vault: <https://sepolia.etherscan.io/address/0x000d8d9C5Ab3b3d98A574eCEB34136e885C16656>

Five settled races are recorded in `evidence/race-*.json`, including a four-tranche race and a single-lock race. Four settled against the currently deployed vault and gate.

The proof path, isolated from our own code

`evidence/proof-path-probe.json` runs the entire Attestcoin path against **three real, pre-existing Sepolia transactions with none of our contracts deployed** — so it tests the protocol integration, not our contracts.

Transaction	Block	calculateTxIndex at 0x0FD2	Explorer	Agrees
0x144bc395f407...	11613749	102	102	yes
0xc58e9177d4de...	11613750	131	131	yes
0xf2b901f98983...	11613751	163	163	yes

Given **nothing but a Merkle path**, the precompile derives the canonical transaction position and it matches what the chain independently reports. This is the protocol's entire thesis reduced to one comparison a sceptic can re-run in a single command.

One shared continuity proof spanning headers 11613749→11613751, **52 continuity roots**, three Merkle proofs, verified off-chain at the precompile for an estimated **3.81×10^{-5} CTC**.

Live precompile report — `evidence/precompile.json`, 10 checks, all passing: both precompiles reachable; Sepolia confirmed as chainKey 1 *read from* 0x0FD3 *itself*; the attestation frontier genuinely lagging Sepolia head by 38 blocks / 8.32 min; a future height correctly **not** attested; `calculateTxIndex` live and deriving index 5 from a three-step path; both the single and batch `verify` selectors decoding; CC3 gas limit 75,000,000, base fee 0.5 gwei, PUSH0 supported.

Honest timing — measured, not asserted

Attestcoin does not document attestation latency anywhere, so we measured it and committed the raw samples.

Stage	Cost
lock lands on Sepolia	instant
attestation reaches that height	6.5-9.3 min
proof generation	seconds
verification at 0x0FD2 + state transition	one Creditcoin block, ~15s

"One block" refers only to the last row. Finality is minutes; settlement is one block once the proof exists. We never claim proving is instant, and the interface is built so it cannot: the wait renders as a live block countdown read from 0x0FD3.

samples	345 , every 60s over a 24-hour window
min · p50 · p90 · p99 · max	6.54 · 7.75 · 8.73 · 9.02 · 9.35 min

Quoted as a **range**, never a mean, because attestation advances in **ten-block batches** — the lag is a sawtooth, and an average describes it wrongly. Two further datasets corroborate the shape: `attestation-vs-finality.jsonl` (70 samples, attestation vs Ethereum's own `safe / finalized`) and `attestation-frontier-gap.jsonl` (41 samples). A direct experiment watching one specific fresh block until `is_height_attested` flips agrees with the indirect proxy to the minute, and shows the mechanism: the frontier runs ~30–40 Sepolia blocks behind head and advances **+10 blocks roughly every 2 minutes**, which is a ~6.6-minute structural floor. Nothing on our side can shorten it.

Deployed contracts

Deployer on both chains: `0xd267C91BF81207222cC8d7300D11CD1849bd98Da`.

Ethereum Sepolia — chainId `11155111`, Attestcoin chainKey `1`

Contract	Address
<code>PriorityVault</code>	<code>0x000d8d9C5Ab3b3d98A574eCEB34136e885C16656</code>
<code>PUSD</code> (settlement token, 6dp)	<code>0xAfA914EF2CF2D754647dc83175Aad6AcF42a9670</code>

Creditcoin CC3 testnet — chainId `102031`

Contract	Address
<code>AttestationGate</code> (the integration core)	<code>0x1E6713DdF4a2D90fb871EF545941b3f190099153</code>
<code>PriorityEngine</code>	<code>0x16FD8A1b7fb4525eFa2D3801F642c84fbb982714</code>
<code>CollateralRegistry</code>	<code>0x39E984ed4875FfC41634e8495B8e45Cc01Ae79b5</code>
<code>ClaimToken</code> (ERC-1155 positions)	<code>0xFc1547aFeb6D69a525780918b3fa40A062372D45</code>
<code>RefinanceEngine</code>	<code>0x07A5b91f866A70D70C3164E3394286E1cB90A433</code>

Addresses come from `contracts/deployments/*.json`, written by the deploy scripts — **never from an env var**, because a mistyped vault address breaks the proof-to-vault binding the security model rests on while looking like a verification failure.

The CC3 explorer is `creditcoin-testnet.blockscout.com`. The host `explorer.cc3-testnet.creditcoin.network` appears in protocol documentation and **does not resolve** — we say so here so nobody reintroduces it, and a test asserts no API response can name it.

Engineering

1,113 tests that need no network, plus 4 fork tests and 10 live-chain precompile checks.

Suite	Tests	What it protects
contracts/test/PriorityProofLib.t.sol	20	one passing rejection per attack
contracts/test/EngineLifecycle.t.sol	28	the settlement lifecycle and the waterfall
contracts/test/PriorityVault.t.sol	25	the source-chain vault, incl. pinned event signatures
contracts/test/AllocationLib.t.sol	13	tranche allocation
contracts/test/AllocationAgreement.t.sol	5	Solidity and TypeScript allocation agree
contracts/test/Toolchain.t.sol	7	the <code>usc-contracts</code> import paths and decoder actually compile
contracts/test/fork/ForkDeploy.t.sol	4	our contracts deploy, wire and settle under real chain conditions
worker/test/proof.test.ts	20	batch flattening — index alignment across three arrays
precedence/test/api/**	692	all 23 API routes
precedence/sdk/test/**	303	ordering, settlement, waterfall, the dApp-side checks

The security controls are tests, not comments. `PriorityProofLib.t.sol` has one *passing rejection* for each of: a look-alike vault, a reverted source transaction, a prover reordering the set, omitting a middle lock, omitting the **first** lock, splicing two races together, a vault whose `seq` contradicts the proof, and the wrong settlement denomination.

All 23 API routes are tested for what they refuse to accept. A lender's rank *is* (`blockHeight`, `txIndex`), so the routes decode those from a receipt fetched by hash rather than accepting them from the party they rank — asserted to hold **even when the request says otherwise**. 422 and 502 are held apart deliberately (the chain answered and refused, versus we could not ask it) because retrying is correct in only one of them.

Nothing in the 1,113 touches a network, on purpose. A unit test that could reach Sepolia would pass or fail on a throttled RPC or an unattested block rather than on the code. The live path is evidenced separately, in `evidence/`.

`@precedence/sdk` — the protocol extracted from the web app as a package with `viem` as its only peer dependency: the data model, the ordering rules, the strict-seniority waterfall, the dApp-side checks and the `0x0FD3` ChainInfo client. It is what someone integrating proven-order priority would import.

Scale: ~37,600 hand-written lines — 2,331 Solidity + 2,124 Solidity tests + 2,305 worker + 20,725 app/SDK + 9,053 app/SDK tests + 1,103 ops. 7 deployed contracts + 3 libraries, 23 API routes, 9 pages, 35 components.

A cross-chain bug we found and fixed, documented in full (`analysis/allocation-divergence.md`): the Creditcoin engine allocated tranche-aware while the Sepolia vault allocated tranche-blind — same money, two rules. A lender could have been paid twice, and a refunded lender could not have recovered their full principal. `AllocationAgreement.t.sol` now asserts the two implementations agree.

The `mock | chain` honesty mechanism. The UI **never** reads an env var to decide what to claim; it reads each adapter's own `isLive()`. Requesting `chain` without deployed addresses does not throw and does not silently pretend — it falls back to fixtures and records **why**, visible at `/api/config`.

Fixtures use `0xSAMPLE...` prefixes, and `ops/check-submission.ts` mechanically enforces that no invented transaction hash, address, block number or explorer link exists anywhere in the repository.

Where the model sits. An LLM is used for exactly one thing: reading an unstructured collateral document and proposing form values, strictly upstream of any capital decision. Bids are deterministic policy — the agent runtime calls the same policy function as the local one — and settlement never depends on a model. Everything a model emits passes through `ratifyExtraction`, which rejects by default, never overwrites the registered record, and reports a document/registration disagreement as a **flag for a human** rather than a correction.

Scope — what the protocol proves

Every boundary here is a design decision, and each one is why a specific part of the system looks the way it does.

Proven ordering, deliberately scoped to ordering. PRECEDENCE proves *which claim is senior* and prevents the same registered claim being financed twice or out of order — the failure that took down Tricolor and First Brands. Whether a custodian issued two receipts for one physical pallet is a *document-authenticity* problem, solved by custody attestation and inspection, and it **composes** with this rather than competing: we do the half that is cryptographically provable, and hash-uniqueness makes the registered identifier itself unforgeable. Keeping the two problems distinct is what lets the proof claim be strong enough to settle capital on.

Cold start is solved. The record exists from the very first financing, because the financing mechanism *is* the registration — no chicken-and-egg, no consortium to assemble, no adoption threshold before the first lender is protected.

Trustless mechanics, portable across jurisdictions. Allocation, seniority, refunds and the unwind all execute without a trusted party. Legal enforceability of a lien remains jurisdiction-specific, which is exactly why the mechanics stand on their own rather than depending on a court to order them.

Readability by design, and the architecture follows from it. Creditcoin proves facts about Sepolia; nothing flows back. That one constraint produces the system's best property: the vault never waits to be told who won. It applies the same deterministic allocation rule Creditcoin applies, so both chains agree on the allocated set with **no message, no bridge and no relayer** between them.

Provenance

Built for this hackathon: all 7 deployed contracts and 3 libraries and their 98 tests; the Attestcoin readability worker; the protocol domain model, state machine, ordering rules and settlement mathematics; every chain adapter, API route and screen in the command centre; the operational and evidence tooling; all 1,113 tests.

Disclosed prior work: the Next.js foundation underneath the command centre — our own design-token system, layout primitives and motion primitives — comes from **Ephera**, an earlier project by this same team, which the team owns outright and reuses on every project it ships. Nothing

protocol-specific is reused: no contract, no proof logic, no settlement rule, no chain adapter, no API route.

Licence: MIT.